Mohammad sameer
ID:B211128
RollNo.:46 (C3)

AI Assignment 2 – Pathfinding Algorithms

This assignment was about finding a path in a binary grid (0 = free cell, 1 = blocked cell). The task was to try two different search algorithms: Best First Search (Greedy) and A* Search, and then compare how they perform. A path can move in 8 directions, and the length is the number of cells visited. I used Manhattan distance as the heuristic because it is simple to calculate and works fine for grid problems.

Part A – Best First Search (Greedy)

In Best First Search, I only used the heuristic function (distance to goal) to decide which node to explore next. This makes it faster but sometimes it doesn't give the shortest path.

```
def best_first_search(grid, heuristic_type="manhattan"):
    n = len(grid)
    start, goal = (0,0), (n-1,n-1)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1, []

    import heapq, math
    def heuristic(a, b):
        if heuristic_type == "euclidean":
            return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
        return abs(a[0]-b[0]) + abs(a[1]-b[1])

    directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
    pq = []
    heapq.heappush(pq, (heuristic(start, goal), start, [start]))
    visited = set([start])

    while pq:
        _, (x,y), path = heapq.heappop(pq)
        if (x,y) == goal:
            return len(path), path
        for dx,dy in directions:
            nx, ny = x+dx, y+dy
            if 0<=nx<n and 0<=ny<n and grid[nx][ny]==0 and (nx,ny) not in visited:
                visited.add((nx,ny))
                heapq.heappush(pq, (heuristic((nx,ny), goal), (nx,ny), path+[(nx,ny)]))
    return -1, []
```

Part B – A* Search

For A* Search, I combined both the cost so far (g) and the heuristic (h). This way, the algorithm is guaranteed to find the shortest path if one exists. It explores more nodes compared to Best First Search but gives the correct answer.

```
def a_star_search(grid, heuristic_type="manhattan"):
    n = len(grid)
    start, goal = (0,0), (n-1,n-1)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1, []

    import heapq, math
    def heuristic(a, b):
        if heuristic_type == "euclidean":
            return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
        return abs(a[0]-b[0]) + abs(a[1]-b[1])

    directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]
    pq = []
    heapq.heappush(pq, (heuristic(start, goal), 0, start, [start]))
    g_score = {start:0}

    while pq:
        f, g, (x,y), path = heapq.heappop(pq)
```

```
        if (x,y) == goal:
            return len(path), path
        for dx,dy in directions:
            nx, ny = x+dx, y+dy
            if 0<=nx<n and 0<=ny<n and grid[nx][ny]==0:
                new_g = g+1
                if (nx,ny) not in g_score or new_g < g_score[(nx,ny)]:
                    g_score[(nx,ny)] = new_g
                    f = new_g + heuristic((nx,ny), goal)
                    heapq.heappush(pq, (f, new_g, (nx,ny), path+[(nx,ny)]))
    return -1, []
```

Example Test Cases

I ran both algorithms on the given examples. Below are the results I got:

```
Example 1:
Grid = [[0,1],[1,0]]
Best First Search → Path length: 2, Path: [(0,0),(1,1)]
A* Search         → Path length: 2, Path: [(0,0),(1,1)]

Example 2:
Grid = [[0,0,0],[1,1,0],[1,1,0]]
Best First Search → Path length: 4, Path: [(0,0),(0,1),(1,2),(2,2)]
A* Search         → Path length: 4, Path: [(0,0),(0,1),(1,2),(2,2)]

Example 3:
Grid = [[1,0,0],[1,1,0],[1,1,0]]
Best First Search → Path length: -1
A* Search         → Path length: -1
```

Comparison (My Observations): - Best First Search was quicker but sometimes it didn't give the shortest path. For example, in some grids it rushed towards the goal and missed better paths. - A* Search always gave the shortest path, but it had to check more cells. So, it is more reliable but a bit slower. In short, Best First is like greedy and fast, but A* is safer if we want the guaranteed shortest path.

Optional Bonus – Visualization

I also tried to visualize A* Search using matplotlib. The idea was to color visited nodes in light blue and the final path in yellow. Obstacles are black and free cells are white. This helped me see how the algorithm expands in different directions before reaching the goal.

```
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import heapq, math

directions = [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]

def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def visualize_grid(grid, path=[], visited=set(), title=""):
    n = len(grid)
    cmap = mcolors.ListedColormap(["white", "black", "lightblue", "yellow"])
    grid_copy = [[grid[i][j] for j in range(n)] for i in range(n)]
    for (x,y) in visited:
        if grid_copy[x][y] == 0:
            grid_copy[x][y] = 2
    for (x,y) in path:
        grid_copy[x][y] = 3
    plt.imshow(grid_copy, cmap=cmap)
    plt.title(title)
    plt.pause(0.5)
    plt.clf()

def a_star_visualize(grid):
    n = len(grid)
    start, goal = (0,0), (n-1,n-1)
    pq = []
    heapq.heappush(pq, (heuristic(start, goal), 0, start, [start]))
    g_score = {start:0}
    visited = set()

    while pq:
        f, g, (x,y), path = heapq.heappop(pq)
        visited.add((x,y))
        visualize_grid(grid, path, visited, title="A* Progress")
        if (x,y) == goal:
            visualize_grid(grid, path, visited, title="Final Path Found")
            return len(path), path
        for dx,dy in directions:
            nx, ny = x+dx, y+dy
            if 0<=nx<n and 0<=ny<n and grid[nx][ny]==0:
                new_g = g+1
                if (nx,ny) not in g_score or new_g < g_score[(nx,ny)]:
                    g_score[(nx,ny)] = new_g
                    f = new_g + heuristic((nx,ny), goal)
                    heapq.heappush(pq, (f, new_g, (nx,ny), path+[(nx,ny)]))
    return -1, []
```

Legend: White = free, Black = obstacle, Light Blue = visited, Yellow = path